

CO4 – ASSESSMENT TOOL 2: INDUSTRY PROBLEM SOLVING TASK

Database Management Systems (CSA05) — MySQL Implementation

CO4: File Organization, Indexing & Hashing Techniques | Weightage: 35% | Total Marks: 50

All answers below are written as executable MySQL (8.0+) scripts — tables, stored procedures/functions, execution plans (EXPLAIN), and analytical verification queries — per the standard laboratory code format.

PRELIMINARY SETUP – Database & Schema Creation

```
-- Create and select working database
CREATE DATABASE IF NOT EXISTS dbms_at2;
USE dbms_at2;

-- Base configuration table for analytical evaluations
CREATE TABLE db_metrics (
    scenario_id    VARCHAR(20) PRIMARY KEY,
    total_records  BIGINT,
    record_size_b  INT,
    block_size_b   INT,
    b_factor       INT
);

INSERT INTO db_metrics VALUES
('E_COMM_10M',    10000000, 200, 4096, FLOOR(4096/200)),
('SUPERMKT_1M',   1000000,  128, 4096, FLOOR(4096/128)),
('HR_5K',         5000,      256, 4096, FLOOR(4096/256)),
('GEN_100K',      100000,   50,  4096, FLOOR(4096/50));

SELECT * FROM db_metrics;
```

Question 1

An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.

```
USE dbms_at2;

CREATE TABLE products (
    product_id     BIGINT PRIMARY KEY,
    sku            VARCHAR(50) UNIQUE,
    title          VARCHAR(150),
    category_id    INT,
    price          DECIMAL(10,2),
    stock_qty      INT,
    status         VARCHAR(20) DEFAULT 'ACTIVE'
) ENGINE=InnoDB;

DELIMITER //
```

```

CREATE PROCEDURE evaluate_ecommerce_storage()
BEGIN
    DECLARE v_records BIGINT DEFAULT 10000000;
    DECLARE v_rec_size INT DEFAULT 200;
    DECLARE v_block_size INT DEFAULT 4096;
    DECLARE v_bfr INT;
    DECLARE v_data_blocks BIGINT;
    DECLARE v_heap_search_io BIGINT;
    DECLARE v_sorted_search_io INT;
    DECLARE v_bplus_search_io INT;

    SET v_bfr = FLOOR(v_block_size / v_rec_size);
    SET v_data_blocks = CEIL(v_records / v_bfr);

    -- Linear scan cost
    SET v_heap_search_io = CEIL(v_data_blocks / 2);
    -- Binary search cost on sorted file
    SET v_sorted_search_io = CEIL(LOG2(v_data_blocks));
    -- B+ Tree index traversal cost (Fanout ~ 256)
    SET v_bplus_search_io = CEIL(LOG(v_records) / LOG(256)) + 1;

    SELECT
        v_records AS total_records,
        v_data_blocks AS total_disk_blocks,
        v_heap_search_io AS heap_avg_search_io,
        v_sorted_search_io AS sorted_binary_search_io,
        v_bplus_search_io AS bplus_tree_search_io,
        'Clustered B+ Tree Organization Recommended' AS recommendation;
END //

DELIMITER ;

CALL evaluate_ecommerce_storage();

```

OUTPUT / TERMINAL RESULT:

```

mysql> CALL evaluate_ecommerce_storage();
+-----+-----+-----+-----+-----+-----+
| total_records | total_disk_blocks | heap_avg_search_io | sorted_binary_search_io | bplus_tree_search_io | recommendation |
+-----+-----+-----+-----+-----+-----+
| 10000000 | 500000 | 250000 | 19 | 4 | Clustered B+ Tree Organization Recommended |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)

```

Question 2

A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.

```
USE dbms_at2;

CREATE TABLE bank_transactions (
    txn_id          BIGINT AUTO_INCREMENT PRIMARY KEY,
    account_no      BIGINT NOT NULL,
    txn_timestamp    DATETIME NOT NULL,
    txn_type         ENUM('DEBIT', 'CREDIT') NOT NULL,
    amount          DECIMAL(15,2) NOT NULL,
    balance_after    DECIMAL(15,2) NOT NULL,
    INDEX idx_acc_date (account_no, txn_timestamp)
) ENGINE=InnoDB;

INSERT INTO bank_transactions (account_no, txn_timestamp, txn_type, amount, balance_after)
VALUES
(1001201, '2026-08-01 10:15:00', 'CREDIT', 15000.00, 65000.00),
(1001201, '2026-08-10 14:20:00', 'DEBIT', 2500.00, 62500.00),
(1001201, '2026-08-15 09:30:00', 'DEBIT', 1200.00, 61300.00),
(1001202, '2026-08-02 11:00:00', 'CREDIT', 8000.00, 28000.00);

-- Query execution plan showing composite index range scan
EXPLAIN SELECT * FROM bank_transactions
WHERE account_no = 1001201
    AND txn_timestamp BETWEEN '2026-08-01 00:00:00' AND '2026-08-15 23:59:59';
```

OUTPUT / TERMINAL RESULT:

```
mysql>
mysql> -- Query execution plan showing composite index range scan
mysql> EXPLAIN SELECT * FROM bank_transactions
-> WHERE account_no = 1001201
-> AND txn_timestamp BETWEEN '2026-08-01 00:00:00' AND '2026-08-15 23:59:59';
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table                | partitions | type | possible_keys | key          | key_len | ref | rows | filtered | Extra              |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1  | SIMPLE      | bank_transactions    | NULL       | range | idx_acc_date  | idx_acc_date | 13      | NULL | 3    | 100.00 | Using index condition |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.01 sec)
```

Question 3

A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.

```
USE dbms_at2;

CREATE TABLE patient_records (
    patient_id      INT PRIMARY KEY,
    doctor_name     VARCHAR(50) NOT NULL,
    patient_name     VARCHAR(50),
    diagnosis        VARCHAR(100),
```

```

        admission_date DATE
    ) ENGINE=InnoDB;

-- Primary clustered index on patient_id (Level 1 + Leaf)
-- Secondary non-clustered B+ Tree index on doctor_name
CREATE INDEX idx_doctor_name ON patient_records(doctor_name);

INSERT INTO patient_records VALUES
(501, 'Dr. Sharma', 'Anand Kumar', 'Hypertension', '2026-08-10'),
(502, 'Dr. Patel', 'Meena Roy', 'Type 2 Diabetes', '2026-08-11'),
(503, 'Dr. Sharma', 'Rajesh V', 'Cardio Arrhythmia', '2026-08-12'),
(504, 'Dr. Varma', 'Kavita S', 'Asthma', '2026-08-14');

-- Verification of execution paths
EXPLAIN SELECT * FROM patient_records WHERE patient_id = 503;
EXPLAIN SELECT * FROM patient_records WHERE doctor_name = 'Dr. Sharma';

```

OUTPUT / TERMINAL RESULT:

```

mysql>
mysql> INSERT INTO patient_records VALUES
-> (501, 'Dr. Sharma', 'Anand Kumar', 'Hypertension', '2026-08-10'),
-> (502, 'Dr. Patel', 'Meena Roy', 'Type 2 Diabetes', '2026-08-11'),
-> (503, 'Dr. Sharma', 'Rajesh V', 'Cardio Arrhythmia', '2026-08-12'),
-> (504, 'Dr. Varma', 'Kavita S', 'Asthma', '2026-08-14');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql>
mysql> -- Verification of execution paths
mysql> EXPLAIN SELECT * FROM patient_records WHERE patient_id = 503;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table      | partitions | type | possible_keys | key      | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1  | SIMPLE     | patient_records | NULL      | const | PRIMARY      | PRIMARY  | 4       | const | 1    | 100.00  | NULL  |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

mysql> EXPLAIN SELECT * FROM patient_records WHERE doctor_name = 'Dr. Sharma';
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table      | partitions | type | possible_keys | key      | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1  | SIMPLE     | patient_records | NULL      | ref  | idx_doctor_name | idx_doctor_name | 202     | const | 2    | 100.00  | NULL  |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

```

Question 4

A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.

```

USE dbms_at2;

CREATE TABLE logistics_eval (
    metric_name          VARCHAR(40),
    btree_metric         VARCHAR(50),
    bplus_tree_metric    VARCHAR(50),
    suitability_verdict   VARCHAR(30)
);

INSERT INTO logistics_eval VALUES

```

```

('Data Record Location', 'Internal and Leaf nodes', 'Leaf nodes exclusively', 'B+ Tree
(High Fan-out)'),
('Node Capacity (Fan-out)', 'Lower (Payload reduces pointers)', 'Higher (Internal holds
only keys)', 'B+ Tree'),
('Range / Tracking Queries', 'In-order tree traversal', 'Linked leaf sequential scan', 'B+
Tree (Optimized)'),
('Insert / Delete Overhead', 'Complex tree rebalancing', 'Standard leaf split / merge', 'B+
Tree (Predictable)'),
('Disk I/O per Query', 'Variable (1 to h)', 'Constant height (h + 1 data I/O)', 'B+ Tree
(Consistent)');

SELECT * FROM logistics_eval;

```

OUTPUT / TERMINAL RESULT:

```

mysql>
mysql> SELECT * FROM logistics_eval;
+-----+-----+-----+-----+
| metric_name          | btree_metric          | bplus_tree_metric     | suitability_verdict    |
+-----+-----+-----+-----+
| Data Record Location | Internal and Leaf nodes | Leaf nodes exclusively | B+ Tree (High Fan-out) |
| Node Capacity (Fan-out) | Lower (Payload reduces pointers) | Higher (Internal holds only keys) | B+ Tree                |
| Range / Tracking Queries | In-order tree traversal | Linked leaf sequential scan | B+ Tree (Optimized)    |
| Insert / Delete Overhead | Complex tree rebalancing | Standard leaf split / merge | B+ Tree (Predictable)  |
| Disk I/O per Query    | Variable (1 to h)      | Constant height (h + 1 data I/O) | B+ Tree (Consistent)  |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)

```

Question 5

Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.

```

USE dbms_at2;

CREATE TABLE hr_hashing_comparison (
    architecture          VARCHAR(30),
    initial_buckets        INT,
    directory_growth       VARCHAR(30),
    overflow_handling       VARCHAR(40),
    scalability_verdict    VARCHAR(40)
);

INSERT INTO hr_hashing_comparison VALUES
('Static Hashing (M=250)', 250, 'Fixed (No Growth)', 'Overflow Chaining (Degrades to
O(N))', 'Prone to high collision'),
('Extendible Hashing', 16, 'Dynamic Doubling (Global Depth)', 'Bucket Splitting on
Local Depth', 'Constant O(1) guaranteed');

DELIMITER //

CREATE PROCEDURE simulate_hr_hash(IN p_emp_id INT)
BEGIN

```

```

DECLARE v_static_bucket INT;
DECLARE v_extendible_dir INT;
DECLARE v_global_depth INT DEFAULT 4; -- 2^4 = 16 initial directory slots

SET v_static_bucket = p_emp_id % 250;
SET v_extendible_dir = p_emp_id % POWER(2, v_global_depth);

SELECT p_emp_id AS employee_id,
       v_static_bucket AS static_bucket_no,
       v_extendible_dir AS extendible_directory_slot,
       v_global_depth AS current_global_depth;
END //

DELIMITER ;

SELECT * FROM hr_hashing_comparison;
CALL simulate_hr_hash(10452);

```

OUTPUT / TERMINAL RESULT:

```

mysql>
mysql> DELIMITER ;
mysql>
mysql> SELECT * FROM hr_hashing_comparison;
Empty set (0.00 sec)

mysql> CALL simulate_hr_hash(10452);
+-----+-----+-----+-----+
| employee_id | static_bucket_no | extendible_directory_slot | current_global_depth |
+-----+-----+-----+-----+
|          10452 |             202 |                4 |                4 |
+-----+-----+-----+-----+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)

```

Question 6

An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.

```

USE dbms_at2;

CREATE TABLE flight_reservations (
    booking_id      BIGINT AUTO_INCREMENT PRIMARY KEY,
    flight_no       VARCHAR(10) NOT NULL,
    flight_date      DATE NOT NULL,
    passenger_name   VARCHAR(50),
    seat_no          VARCHAR(5),
    fare_paid        DECIMAL(10,2),
    INDEX idx_flight_date (flight_no, flight_date)

```

```

) ENGINE=InnoDB;

INSERT INTO flight_reservations (flight_no, flight_date, passenger_name, seat_no,
fare_paid) VALUES
('AI-204', '2026-09-01', 'Dheeraj Krushna', '12A', 4500.00),
('AI-204', '2026-09-05', 'Suresh Raina', '14C', 5200.00),
('AI-204', '2026-09-12', 'Venkata Krishna', '12B', 4800.00),
('6E-512', '2026-09-01', 'Kavita Roy', '03F', 3900.00);

-- Point Query by flight_no + Range Query by flight_date
EXPLAIN SELECT * FROM flight_reservations
WHERE flight_no = 'AI-204'
AND flight_date BETWEEN '2026-09-01' AND '2026-09-10';

```

OUTPUT / TERMINAL RESULT:

```

mysql>
mysql> INSERT INTO flight_reservations (flight_no, flight_date, passenger_name, seat_no, fare_paid) VALUES
-> ('AI-204', '2026-09-01', 'Dheeraj Krushna', '12A', 4500.00),
-> ('AI-204', '2026-09-05', 'Suresh Raina', '14C', 5200.00),
-> ('AI-204', '2026-09-12', 'Venkata Krishna', '12B', 4800.00),
-> ('6E-512', '2026-09-01', 'Kavita Roy', '03F', 3900.00);
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql>
mysql> -- Point Query by flight_no + Range Query by flight_date
mysql> EXPLAIN SELECT * FROM flight_reservations
-> WHERE flight_no = 'AI-204'
-> AND flight_date BETWEEN '2026-09-01' AND '2026-09-10';
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table                | partitions | type | possible_keys | key                | key_len | ref | rows | filtered | Extra                |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE      | flight_reservations | NULL      | range | idx_flight_date | idx_flight_date    | 45      | NULL | 2    | 100.00 | Using index condition |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

```

Question 7

For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.

```

USE dbms_at2;

CREATE TABLE university_students (
    student_id    INT PRIMARY KEY,                -- Primary clustered index
    name          VARCHAR(50),
    department    VARCHAR(30) NOT NULL,
    cgpa          DECIMAL(3,2) NOT NULL,
    admission_year INT,
    INDEX idx_dept_cgpa (department, cgpa)        -- Composite secondary index
) ENGINE=InnoDB;

INSERT INTO university_students VALUES
(101, 'Arun', 'AI-ML', 8.95, 2025),
(102, 'Bala', 'AI-ML', 9.40, 2025),
(103, 'Chitra', 'CSE', 7.80, 2025),

```

```

(104, 'Deepa', 'AI-ML', 8.20, 2025),
(105, 'Elango', 'ECE', 8.50, 2025);

-- Fast filtered range query on secondary index
EXPLAIN SELECT student_id, name, cgpa FROM university_students
WHERE department = 'AI-ML' AND cgpa >= 8.50;

```

OUTPUT / TERMINAL RESULT:

```

mysql>
mysql> INSERT INTO university_students VALUES
-> (101, 'Arun', 'AI-ML', 8.95, 2025),
-> (102, 'Bala', 'AI-ML', 9.40, 2025),
-> (103, 'Chitra', 'CSE', 7.80, 2025),
-> (104, 'Deepa', 'AI-ML', 8.20, 2025),
-> (105, 'Elango', 'ECE', 8.50, 2025);
Query OK, 5 rows affected (0.01 sec)
Records: 5 Duplicates: 0 Warnings: 0

mysql>
mysql> -- Fast filtered range query on secondary index
mysql> EXPLAIN SELECT student_id, name, cgpa FROM university_students
-> WHERE department = 'AI-ML' AND cgpa >= 8.50;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | university_students | NULL | range | idx_dept_cgpa | idx_dept_cgpa | 124 | NULL | 2 | 100.00 | Using index condition |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

```

Question 8

Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.

```

USE dbms_at2;

DELIMITER //

CREATE PROCEDURE calculate_block_utilization()
BEGIN
    DECLARE v_rec_count BIGINT DEFAULT 100000;
    DECLARE v_rec_size INT DEFAULT 50;
    DECLARE v_block_size INT DEFAULT 4096;
    DECLARE v_bfr INT;
    DECLARE v_heap_blocks INT;
    DECLARE v_heap_util DECIMAL(5,2);
    DECLARE v_hash_util DECIMAL(5,2);
    DECLARE v_hash_blocks INT;

    SET v_bfr = FLOOR(v_block_size / v_rec_size);
    SET v_heap_blocks = CEIL(v_rec_count / v_bfr);
    SET v_heap_util = ((v_bfr * v_rec_size) / v_block_size) * 100;

    -- Hash utilization assuming 70% loading factor to minimize bucket overflow
    SET v_hash_blocks = CEIL(v_heap_blocks / 0.70);
    SET v_hash_util = (v_heap_util * 0.70);

    SELECT

```



```

        'Heap / Sorted Organization' AS org_type,
        v_rec_count AS records,
        v_bfr AS blocking_factor,
        v_heap_blocks AS total_blocks_allocated,
        v_heap_util AS block_utilization_pct
    UNION ALL
    SELECT
        'Hashed Organization (70% Target Load)',
        v_rec_count,
        v_bfr,
        v_hash_blocks,
        v_hash_util;
END //

DELIMITER ;

CALL calculate_block_utilization();

```

OUTPUT / TERMINAL RESULT:

```

mysql>
mysql> DELIMITER ;
mysql>
mysql> CALL calculate_block_utilization();
+-----+-----+-----+-----+-----+
| org_type                                | records | blocking_factor | total_blocks_allocated | block_utilization_pct |
+-----+-----+-----+-----+-----+
| Heap / Sorted Organization              | 100000  | 81              | 1235                   | 98.88                 |
| Hashed Organization (70% Target Load) | 100000  | 81              | 1765                   | 69.22                 |
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

```

Question 9

A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.

```

USE dbms_at2;

CREATE TABLE social_posts (
    post_id          BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id          BIGINT NOT NULL,
    created_at       DATETIME NOT NULL,
    hashtag          VARCHAR(50) NOT NULL,
    content          VARCHAR(280),
    INDEX idx_user_feed (user_id, created_at DESC),
    INDEX idx_hashtag_feed (hashtag, created_at DESC)
) ENGINE=InnoDB;

INSERT INTO social_posts (user_id, created_at, hashtag, content) VALUES
(8801, '2026-08-25 10:00:00', 'TechTrends', 'Exploring B+ Tree implementations!'),
(8801, '2026-08-25 11:30:00', 'AI', 'Evaluating LLM indexing algorithms'),

```

```
(8802, '2026-08-25 11:45:00', 'TechTrends', 'Database storage engine optimization');
```

```
-- Query 1: User profile timeline scan
```

```
EXPLAIN SELECT * FROM social_posts
```

```
WHERE user_id = 8801
```

```
ORDER BY created_at DESC
```

```
LIMIT 10;
```

```
-- Query 2: Trending topic timeline scan
```

```
EXPLAIN SELECT * FROM social_posts
```

```
WHERE hashtag = 'TechTrends'
```

```
ORDER BY created_at DESC
```

```
LIMIT 10;
```

OUTPUT / TERMINAL RESULT:

```
mysql>
mysql> -- Query 1: User profile timeline scan
mysql> EXPLAIN SELECT * FROM social_posts
-> WHERE user_id = 8801
-> ORDER BY created_at DESC
-> LIMIT 10;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table      | partitions | type | possible_keys | key           | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1  | SIMPLE      | social_posts | NULL       | ref  | idx_user_feed | idx_user_feed | 8       | const | 2    | 100.00  | NULL  |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

mysql>
mysql> -- Query 2: Trending topic timeline scan
mysql> EXPLAIN SELECT * FROM social_posts
-> WHERE hashtag = 'TechTrends'
-> ORDER BY created_at DESC
-> LIMIT 10;
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table      | partitions | type | possible_keys | key           | key_len | ref | rows | filtered | Extra |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1  | SIMPLE      | social_posts | NULL       | ref  | idx_hashtag_feed | idx_hashtag_feed | 202     | const | 2    | 100.00  | NULL  |
+----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

Question 10

Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.

```
USE dbms_at2;
```

```
CREATE TABLE file_org_performance (
    file_organization    VARCHAR(30) PRIMARY KEY,
    insert_cost           VARCHAR(30),
    delete_cost          VARCHAR(30),
    exact_search_cost     VARCHAR(30),
    range_search_cost     VARCHAR(30),
    suitability_verdict   VARCHAR(50)
);
```

```
INSERT INTO file_org_performance VALUES
('Heap File', 'O(1) [Append to end]', 'O(B) [Scan + Flag update]', 'O(B) [Full file
```

```
scan]', 'O(B) [Full file scan]', 'Best for logging/insert speed; poor query'),
('Sorted File', 'O(B) [Reorganize file]', 'O(B) [Shift/Defrag]', 'O(log2 B) [Binary
search]', 'O(log2 B + Range blocks)', 'Slow continuous billing writes'),
('Hashed File', 'O(1) [Direct bucket]', 'O(1) [Direct bucket]', 'O(1) [Hash lookup]',
'O(B) [Inefficient scan]', 'Optimal for POS item code lookups');

SELECT * FROM file_org_performance;
```

OUTPUT / TERMINAL RESULT:

```
-- range_search_cost VARCHAR(30),
-- suitability_verdict VARCHAR(50)
-- );
Query OK, 0 rows affected (0.06 sec)

mysql>
mysql> INSERT INTO file_org_performance VALUES
-- ('Heap File', 'O(1) [Append to end]', 'O(B) [Scan + Flag update]', 'O(B) [Full file scan]', 'O(B) [Full file scan]', 'Best for logging/insert speed; poor query'),
-- ('Sorted File', 'O(B) [Reorganize file]', 'O(B) [Shift/Defrag]', 'O(log2 B) [Binary search]', 'O(log2 B + Range blocks)', 'Slow continuous billing writes'),
-- ('Hashed File', 'O(1) [Direct bucket]', 'O(1) [Direct bucket]', 'O(1) [Hash lookup]', 'O(B) [Inefficient scan]', 'Optimal for POS item code lookups');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql>
mysql> SELECT * FROM file_org_performance;
+-----+-----+-----+-----+-----+-----+
| file_organization | insert_cost | delete_cost | exact_search_cost | range_search_cost | suitability_verdict |
+-----+-----+-----+-----+-----+-----+
| Hashed File      | O(1) [Direct bucket] | O(1) [Direct bucket] | O(1) [Hash lookup] | O(B) [Inefficient scan] | Optimal for POS item code lookups |
| Heap File        | O(1) [Append to end] | O(B) [Scan + Flag update] | O(B) [Full file scan] | O(B) [Full file scan] | Best for logging/insert speed; poor query |
| Sorted File      | O(B) [Reorganize file] | O(B) [Shift/Defrag] | O(log2 B) [Binary search] | O(log2 B + Range blocks) | Slow continuous billing writes |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> |
```